

Développement d'un site web dynamique

MySQL et PHP



On parle de quoi ?

- 1 Introduction aux bases de données
- 2 Connexion à MySQL avec PHP
- 3 Exécuter des requêtes SQL
- 4 Récupérer et afficher les résultats
- 5 Sécurité : les injections SQL

Pourquoi une base de données ?



Stocker des données de manière permanente

Une base de données permet de conserver les informations même après la fin du script.

1

Utilisateurs

Logins, mots de passe, profils

2

Produits

Catalogue, prix, stock

3

Commandes

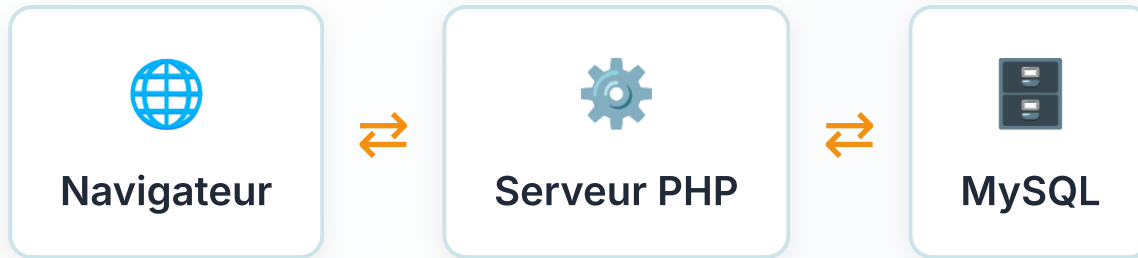
Historique des achats

4

Articles

Blog, actualités, contenu

Architecture Client-Serveur-BDD



Le navigateur envoie une requête au serveur

PHP traite la requête et interroge la base de données

MySQL renvoie les données à PHP

PHP génère le HTML et l'envoie au navigateur

Les étapes en PHP

1

Connexion

Se connecter à la base de données MySQL

2

Requête

Envoyer une requête SQL

3

Résultat

Récupérer et traiter les données

4

Fermeture

Fermer la connexion

Étape 1 : Connexion à MySQL

```
<?php
$connexion = mysqli_connect("localhost", "utilisateur", "motdepasse", "nom_base");

if (!$connexion) {
    die("Erreur de connexion : " . mysqli_connect_error());
}

echo "Connexion réussie !";
?>
```



mysqli_connect()

Établit une connexion avec un serveur MySQL. Retourne un objet de connexion en cas de succès, ou **false** en cas d'échec. En cas d'échec, **mysqli_connect_error()** retourne le message d'erreur.



Paramètres de connexion

localhost, utilisateur, mot de passe et nom de la base doivent correspondre à votre configuration MySQL.

mysqli_connect() - Paramètres

Paramètre	Description	Exemple
<code>serveur</code>	Adresse du serveur MySQL	"localhost"
<code>utilisateur</code>	Nom d'utilisateur MySQL	"root"
<code>motdepasse</code>	Mot de passe de l'utilisateur	"monmdp123"
<code>base</code>	Nom de la base de données	"ma_boutique"

Étape 2 : Exécuter une requête SELECT

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$sql = "SELECT * FROM produits";
$resultat = mysqli_query($connexion, $sql);

if ($resultat) {
    echo "Requête exécutée avec succès !";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}
?>
```



mysqli_query()

Envoie une requête SQL au serveur MySQL. Retourne un objet résultat pour SELECT, ou **true**/**false** pour INSERT, UPDATE, DELETE.

Étape 3 : Récupérer les résultats

```
<?php
$sql = "SELECT * FROM produits";
$resultat = mysqli_query($connexion, $sql);

// Parcourir tous les résultats
while ($ligne = mysqli_fetch_assoc($resultat)) {
    echo $ligne['nom'] . " - " . $ligne['prix'] . " euros<br>";
}
?>
```



mysqli_fetch_assoc()

Récupère une ligne du résultat sous forme de tableau associatif. Retourne **NULL** quand il n'y a plus de lignes.

Exemple : Afficher dans un tableau HTML

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");
$resultat = mysqli_query($connexion, "SELECT * FROM produits");
?>
<table border="1">
  <tr>
    <th>ID</th>
    <th>Nom</th>
    <th>Prix</th>
  </tr>
  <?php
  while ($produit = mysqli_fetch_assoc($resultat)) {
    echo "<tr>";
    echo "<td>" . $produit['id'] . "</td>";
    echo "<td>" . $produit['nom'] . "</td>";
    echo "<td>" . $produit['prix'] . " euros</td>";
    echo "</tr>";
  }
?>
</table>
```

Étape 4 : Fermer la connexion

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

// ... vos requêtes ...

mysqli_close($connexion);
?>
```



mysqli_close()

Ferme la connexion au serveur MySQL et libère les ressources associées. Bonne pratique : toujours l'appeler à la fin du script.

Exemple complet

```
<?php
// 1. Connexion
$connexion = mysqli_connect("localhost", "root", "", "boutique");
if (!$connexion) {
    die("Erreur : " . mysqli_connect_error());
}
// 2. Requête
$sql = "SELECT * FROM produits WHERE prix < 100";
$resultat = mysqli_query($connexion, $sql);
// 3. Affichage des résultats
if (mysqli_num_rows($resultat) > 0) {
    while ($produit = mysqli_fetch_assoc($resultat)) {
        echo $produit['nom'] . " : " . $produit['prix'] . " euros<br>";
    }
} else {
    echo "Aucun produit trouvé.";
}
// 4. Fermeture
mysqli_close($connexion);
?>
```


Requête INSERT

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$sql = "INSERT INTO produits (nom, prix) VALUES ('T-shirt', 29.99)";

if (mysqli_query($connexion, $sql)) {
    echo "Produit ajouté avec succès !";
    echo "ID du nouveau produit : " . mysqli_insert_id($connexion);
} else {
    echo "Erreur : " . mysqli_error($connexion);
}

mysqli_close($connexion);
?>
```

Requête UPDATE

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$sql = "UPDATE produits SET prix = 24.99 WHERE id = 5";

if (mysqli_query($connexion, $sql)) {
    echo "Produit mis à jour !";
    echo mysqli_affected_rows($connexion) . " ligne(s) modifiée(s)";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}

mysqli_close($connexion);
?>
```

Requête DELETE

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$sql = "DELETE FROM produits WHERE id = 5";

if (mysqli_query($connexion, $sql)) {
    echo "Produit supprimé !";
    echo mysqli_affected_rows($connexion) . " ligne(s) supprimée(s)";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}

mysqli_close($connexion);
?>
```

Fonctions utiles

 **mysqli_num_rows()**

Nombre de lignes retournées par SELECT

 **mysqli_affected_rows()**

Nombre de lignes affectées par
INSERT/UPDATE/DELETE

 **mysqli_insert_id()**

ID auto-incrémenté du dernier INSERT

 **mysqli_error()**

Message d'erreur de la dernière requête

Les injections SQL : une menace réelle

Tech

Yahoo investigating exposure of 400,000 passwords

12 July 2012



Yahoo is looking into claims a hacker has stolen 400,000 passwords



Qu'est-ce qu'une injection SQL ?

Une attaque où un utilisateur malveillant **insère du code SQL** dans un champ de formulaire pour manipuler la requête exécutée par le serveur.



Yahoo, 2012

Un hacker a extrait **400 000 identifiants et mots de passe** via une injection SQL. Les mots de passe étaient stockés **en clair**.



Source : BBC News



Ce type d'attaque est encore l'une des plus répandues aujourd'hui.

Danger : Injection SQL

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

// ⚠ La variable vient directement du formulaire !
$email = $_POST['email'];
$mdp   = $_POST['mdp'];

$sql = "SELECT * FROM utilisateurs WHERE email = '$email' AND mdp = '$mdp'";
$resultat = mysqli_query($connexion, $sql);

if (mysqli_num_rows($resultat) > 0) {
    echo "Connecté !"; // ← l'attaquant arrive ici
} else {
    echo "Email ou mot de passe incorrect.";
}
?>

<form method="POST">
    <input type="text"      name="email" placeholder="Email">
    <input type="password" name="mdp"   placeholder="Mot de passe">
    <button type="submit">Se connecter</button>
</form>
```

Injection SQL — Comment ça marche ?

Ce que l'attaquant entre

Email :

' OR '1'='1' #

Mot de passe : *(n'importe quoi)*

blabla

Requête générée

```
SELECT * FROM utilisateurs  
WHERE email = '' OR '1'='1' #  
AND mdp = 'blabla'
```

Le # est un commentaire SQL (MySQL/MariaDB) : tout ce qui suit est **ignoré**, y compris la vérification du mot de passe.

Résultat

'1'='1' est toujours vrai → tous les utilisateurs sont retournés → `mysqli_num_rows() > 0` → l'attaquant est connecté en tant qu'administrateur sans connaître aucun mot de passe.

Solution : Requêtes préparées

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

// Les ? remplacent les variables – jamais de variables directement !
$sql = "SELECT * FROM utilisateurs WHERE email = ? AND mdp = ?";
$stmt = mysqli_prepare($connexion, $sql);

// Lier les paramètres ("ss" = 2 strings)
mysqli_stmt_bind_param($stmt, "ss", $_POST['email'], $_POST['mdp']);

// Exécuter et récupérer le résultat
mysqli_stmt_execute($stmt);
$resultat = mysqli_stmt_get_result($stmt);

if (mysqli_num_rows($resultat) > 0) {
    echo "Connecté !";
} else {
    echo "Email ou mot de passe incorrect.";
}
?>
```

Pourquoi c'est sécurisé ?

Les **?** sont des emplacements réservés. MySQL reçoit la requête et les données **séparément** — les données ne peuvent jamais modifier la structure de la requête.

Types de paramètres

Lettre	Type	Exemple
s	String (chaîne)	"alice@example.com"
i	Integer (entier)	42
d	Double (décimal)	19.99
b	Blob (binaire)	Fichiers, images

```
// Exemples
mysqli_stmt_bind_param($stmt, "s", $email);           // 1 string
mysqli_stmt_bind_param($stmt, "si", $nom, $id);       // 1 string, 1 int
mysqli_stmt_bind_param($stmt, "ssd", $nom, $email, $prix); // 2 strings, 1 double
```

SELECT avec requête préparée

/ plusieurs résultats

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$categorie = $_GET['categorie'];

// Préparer la requête SELECT
$sql = "SELECT * FROM produits WHERE categorie = ?";
$stmt = mysqli_prepare($connexion, $sql);

// Lier le paramètre (s = string)
mysqli_stmt_bind_param($stmt, "s", $categorie);

// Exécuter et récupérer le résultat
mysqli_stmt_execute($stmt);
$resultat = mysqli_stmt_get_result($stmt);

// Parcourir toutes les lignes avec une boucle while
while ($ligne = mysqli_fetch_assoc($resultat)) {
    echo $ligne['nom'] . " — " . $ligne['prix'] . " €<br>";
}

mysqli_stmt_close($stmt);
mysqli_close($connexion);
?>
```

SELECT avec requête préparée

/ un seul résultat

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$id = $_GET['id'];

// Préparer la requête SELECT (recherche par clé primaire → 1 seul résultat)
$sql = "SELECT * FROM produits WHERE id = ?";
$stmt = mysqli_prepare($connexion, $sql);

// Lier le paramètre (i = integer)
mysqli_stmt_bind_param($stmt, "i", $id);

// Exécuter et récupérer le résultat
mysqli_stmt_execute($stmt);
$resultat = mysqli_stmt_get_result($stmt);

// Récupérer l'unique ligne directement
$ligne = mysqli_fetch_assoc($resultat);

if ($ligne) {
    echo $ligne['nom'] . " - " . $ligne['prix'] . " €";
} else {
    echo "Produit introuvable.";
}

mysqli_stmt_close($stmt);
mysqli_close($connexion);
?>
```

INSERT avec requête préparée

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$nom = "T-shirt";
$prix = 29.99;

// Préparer la requête INSERT
$sql = "INSERT INTO produits (nom, prix) VALUES (?, ?)";
$stmt = mysqli_prepare($connexion, $sql);

// Lier les paramètres (sd = string, double)
mysqli_stmt_bind_param($stmt, "sd", $nom, $prix);

// Exécuter
if (mysqli_stmt_execute($stmt)) {
    echo "Produit ajouté ! ID : " . mysqli_insert_id($connexion);
}

mysqli_stmt_close($stmt);
mysqli_close($connexion);
?>
```

UPDATE avec requête préparée

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$nouveau_prix = 24.99;
$id = 1;

// Préparer la requête UPDATE
$sql = "UPDATE produits SET prix = ? WHERE id = ?";
$stmt = mysqli_prepare($connexion, $sql);

// Lier les paramètres (di = double, integer)
mysqli_stmt_bind_param($stmt, "di", $nouveau_prix, $id);

// Exécuter
if (mysqli_stmt_execute($stmt)) {
    echo mysqli_stmt_affected_rows($stmt) . " ligne(s) modifiée(s)";
}

mysqli_stmt_close($stmt);
mysqli_close($connexion);
?>
```

DELETE avec requête préparée

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");

$id = 5;

// Préparer la requête DELETE
$sql = "DELETE FROM produits WHERE id = ?";
$stmt = mysqli_prepare($connexion, $sql);

// Lier le paramètre (i = integer)
mysqli_stmt_bind_param($stmt, "i", $id);

// Exécuter
if (mysqli_stmt_execute($stmt)) {
    echo mysqli_stmt_affected_rows($stmt) . " ligne(s) supprimée(s)";
}

mysqli_stmt_close($stmt);
mysqli_close($connexion);
?>
```

Exemple complet avec formulaire

```
<?php
$connexion = mysqli_connect("localhost", "root", "", "boutique");
if (isset($_POST['nom'])) {
    // Préparer la requête
    $sql = "INSERT INTO produits (nom, prix) VALUES (?, ?)";
    $stmt = mysqli_prepare($connexion, $sql);
    // Lier les paramètres
    mysqli_stmt_bind_param($stmt, "sd", $_POST['nom'], $_POST['prix']);
    if (mysqli_stmt_execute($stmt)) {
        echo "Produit ajouté !";
    }
    mysqli_stmt_close($stmt);
}
mysqli_close($connexion);
?>

<form method="POST" action="">
    <input type="text" name="nom" placeholder="Nom du produit">
    <input type="number" name="prix" step="0.01" placeholder="Prix">
    <button type="submit">Ajouter</button>
</form>
```

Se protéger des injections SQL

1

Requêtes préparées

Ne jamais insérer de variables directement dans une requête. Utiliser `mysqli_prepare()` et les placeholders `?`.

2

Valider les données

Vérifier le type et le format des données reçues avant de les utiliser (`is_numeric()`, `filter_var()`...).

3

Limiter les droits

L'utilisateur MySQL de l'application ne doit avoir que les droits strictement nécessaires (pas de `DROP`, pas de `CREATE`...).

4

Masquer les erreurs

Ne jamais afficher les erreurs SQL en production — elles donnent des informations précieuses à un attaquant.

La règle d'or

Ne jamais faire confiance aux données venant de l'utilisateur. Toujours valider, toujours utiliser les requêtes préparées.

Récapitulatif — Fonctions communes

Fonction	Utilité
<code>mysqli_connect()</code>	Ouvrir une connexion
<code>mysqli_connect_error()</code>	Message d'erreur de connexion
<code>mysqli_fetch_assoc()</code>	Récupérer une ligne du résultat
<code>mysqli_insert_id()</code>	ID du dernier INSERT
<code>mysqli_close()</code>	Fermer la connexion

Récapitulatif — Requêtes non préparées

Fonction	Utilité
<code>mysqli_query()</code>	Exécuter une requête SQL
<code>mysqli_num_rows()</code>	Nombre de lignes retournées (SELECT)
<code>mysqli_affected_rows()</code>	Lignes affectées (INSERT/UPDATE/DELETE)
<code>mysqli_error()</code>	Message d'erreur de la dernière requête

Récapitulatif — Requêtes préparées

Fonction	Utilité
<code>mysqli_prepare()</code>	Préparer une requête sécurisée
<code>mysqli_stmt_bind_param()</code>	Lier les paramètres à la requête
<code>mysqli_stmt_execute()</code>	Exécuter la requête préparée
<code>mysqli_stmt_get_result()</code>	Récupérer le résultat d'un SELECT
<code>mysqli_stmt_affected_rows()</code>	Lignes affectées (INSERT/UPDATE/DELETE)
<code>mysqli_stmt_close()</code>	Fermer le statement

Bonnes pratiques

- 1 **Toujours** vérifier si la connexion a réussi
- 2 **Toujours** utiliser les requêtes préparées avec des données utilisateur
- 3 **Toujours** fermer les statements avec `mysqli_stmt_close()`
- 4 **Toujours** fermer la connexion à la fin avec `mysqli_close()`
- 5 **Ne jamais** afficher les erreurs SQL en production